Java para a OBI

Apostila, exercícios e gabaritos para 4 dias de treino

Plano de treino

Dia	Faça	Meta
1	Recordes e Maior sequência	Ler a entrada e resolver cada questão em uma passada
2	Mediana e Frequências	Praticar ordenação, vetores e contagem
3	Somas de intervalos	Comparar uma solução direta com soma de prefixos
4	Maratona 2022: problemas I, A, J, E, D e F	Resolver problemas reais de prova, do mais fácil ao mais difícil

Tente cada questão antes de olhar o gabarito.

Nome

Nível / modalidade

Data da prova

Sumário

Parte 1 — Apostila

1. Estrutura de um programa
2. Como declarar e mudar valores
3. Como pegar valores da entrada
4. Condições e laços
5. Vetores, matrizes e strings
6. Métodos
7. Biblioteca que vale decorar
8. Padrões de prova
9. Antes de submeter

Parte 2 — Exercícios e gabaritos

Exercícios com espaço para rascunho
Gabaritos comentados

Parte 3 — Para levar

Folha de consulta rápida
Registro de tentativas e checklist

Como rodar no computador

```
javac Main.java
"3 8" | java Main            # PowerShell
echo "3 8" | java Main       # Linux / macOS / Git Bash
```

Parte 1 — Apostila

Os comentários nos códigos mostram o valor de cada variável no exemplo.

Use esta apostila junto com os exercícios (Parte 2). Estude as seções 1 a 8 e consulte a seção 9 antes de submeter.

1. Estrutura de um programa

Salve este exemplo como `Main.java` para rodar localmente.

> **Na Maratona:** o arquivo tem o nome do problema. Problema A → arquivo `A.java` com `public class A`.

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int a = entrada.nextInt();   // a = 3
        int b = entrada.nextInt();   // b = 8
        System.out.println(a + b);   // imprime 11
    }
}
```

Com a entrada `3 8`, a saída é `11`. Você também pode digitar um número por linha.

- `import` permite usar a classe `Scanner` pelo nome curto.
- `class Main` define a classe. Uma classe pública precisa ter o mesmo nome do arquivo.
- `main` é onde a execução começa. `void` indica que o método não devolve um valor.
- Chaves delimitam blocos. Declarações, atribuições e chamadas como `println` terminam com ponto e vírgula.
- `//` inicia um comentário de uma linha. `/* ... */` delimita um comentário de várias linhas.
- Java distingue maiúsculas de minúsculas. `String` e `string` são nomes diferentes.

```
javac Main.java
"3 8" | java Main
```

2. Como declarar e mudar valores

O padrão é `tipo nome = valor;`. Declare uma vez e use o nome para ler ou alterar o valor.

```java
int pontos = 10;          // pontos = 10
pontos = pontos + 5;      // pontos agora vale 15
pontos += 2;              // 17
pontos++;                 // 18
final int LIMITE = 100;   // constante
```

Tipo	Exemplo	Uso
`int`	`int n = 42;`	Inteiros de -2^{31} até $2^{31}-1$
`long`	`long total = 3_000_000_000L;`	Inteiros de -2^{63} até $2^{63}-1$
`double`	`double media = 7.5;`	Valores com parte decimal
`boolean`	`boolean achou = false;`	Verdadeiro ou falso
`char`	`char letra = 'A';`	Um caractere simples, com aspas simples
`String`	`String nome = "Ana";`	Texto, com aspas duplas

Uma variável local precisa receber um valor antes de ser lida. Uma variável declarada dentro de um bloco só existe naquele bloco. O índice declarado em `for (int i = 0; ...)` fica disponível dentro do laço.

Operadores e conversões

Sintaxe	Significado
`+ - * / %`	Soma, diferença, produto, divisão e resto
`= += -= *= /=`	Atribuição e atualização
`== != < <= > >=`	Comparações que resultam em `boolean`
`&&`, `||`, `!`	E, ou, negação
`condicao ? a : b`	Escolhe `a` se a condição for verdadeira, ou `b` caso contrário

```java
int quociente = 7 / 2;           // 3
double metade = 7 / 2.0;         // 3.5
int resto = 7 % 2;               // 1
int truncado = (int) 3.9;        // 3
int numero = Integer.parseInt("42"); // 42
long produto = 1L * 100_000 * 100_000; // 10.000.000.000
int maior = Math.max(12, 8);     // 12
```

Em um produto de dois `int`, a operação acontece em `int`, mesmo se o resultado for atribuído a um `long`. Converta um dos operandos antes de multiplicar. Para ordenar ou comparar inteiros, prefira `Integer.compare(a, b)` a `a - b`, que pode estourar.

3. Como pegar valores da entrada

Adicione `import java.util.Scanner;` no topo e crie um único leitor dentro de `main`.

```java
Scanner entrada = new Scanner(System.in);
int idade = entrada.nextInt();        // 15
long populacao = entrada.nextLong();  // 3000000000
String palavra = entrada.next();      // "Ana"
char inicial = palavra.charAt(0);     // 'A'
```

A entrada `15 3000000000 Ana` fornece `15`, `3000000000`, `"Ana"` e `'A'`. Esses métodos leem tokens separados por espaços, tabulações ou quebras de linha.

Quero ler	Método
Inteiro	`nextInt()`
Inteiro grande	`nextLong()`
Decimal	`nextDouble()`
Uma palavra	`next()`
Uma linha inteira	`nextLine()`
Vários valores até acabar a entrada	`while (entrada.hasNextInt()) { ... }`

Para decimais com ponto, adicione `import java.util.Locale;` e chame `entrada.useLocale(Locale.US);` antes de ler.

Se o enunciado dá um inteiro sozinho na primeira linha e um texto na segunda, como `2` e `Ola mundo`:

```
int n = entrada.nextInt();          // 2
entrada.nextLine();                 // devolve "", o resto da primeira linha
String frase = entrada.nextLine(); // "Ola mundo", a segunda linha com espaços
```

Se o texto estiver na mesma linha do inteiro, a primeira chamada a `nextLine()` já devolve esse texto, incluindo o espaço separador.

Como imprimir

```
System.out.println(42);           // 42 e quebra de linha
System.out.print("3 ");           // sem quebra de linha
System.out.println(8);            // completa a linha "3 8"
System.out.printf(Locale.US, "%.2f%n", 3.5); // 3.50
```

Em uma submissão, imprima só o que o enunciado pede. Para muitas respostas, use `StringBuilder` e imprima uma vez ao final.

4. Condições e laços

```
int nota = 7;
if (nota >= 7) {
    System.out.println("Aprovado");    // este ramo executa, pois 7 >= 7
} else if (nota >= 5) {
    System.out.println("Recuperacao");
} else {
    System.out.println("Reprovado");
}
```

```java
int soma = 0;
for (int i = 1; i <= 5; i++) { // início, condição, avanço
    soma += i;                 // soma: 1, 3, 6, 10, 15
}
// soma = 15

int faltam = 3;
while (faltam > 0) {
    faltam--;                  // faltam: 2, 1, 0
}
// faltam = 0

for (int i = 0; i < 10; i++) {
    if (i == 8) {
        break;             // sai do laço
    }
    if (i % 2 != 0) {
        continue;          // pula o restante desta iteração
    }
    System.out.println(i); // 0, 2, 4, 6, um por linha
}
```

Em `a && b`, Java só avalia `b` se `a` for verdadeiro. Isso permite escrever `i < v.length && v[i] == alvo` quando `i` é não negativo.

5. Vetores, matrizes e strings

```java
int[] v = new int[3];  // começa como {0, 0, 0}
v[0] = 9;              // v = {9, 0, 0}
int primeiro = v[0];   // 9
int ultimo = v[v.length - 1]; // 0; exige pelo menos um elemento

int[] dados = {9, -2, 4};
for (int i = 0; i < dados.length; i++) {
    System.out.println(dados[i]); // i = 0, 1, 2 imprime 9, -2, 4
}
for (int valor : dados) { // quando você só precisa do valor
    System.out.println(valor);    // valor = 9, -2, 4
}
```

Ler N números e guardar no vetor

```java
int n = entrada.nextInt();        // 3
int[] valores = new int[n];       // [0, 0, 0]
for (int i = 0; i < n; i++) {
    valores[i] = entrada.nextInt(); // [8, 0, 0], [8, 4, 0], [8, 4, 9]
}
int primeiraPosicao = valores[0]; // 8; exige n > 0
int posicaoDoEnunciado = 2;
int valorNaPosicao = valores[posicaoDoEnunciado - 1]; // valores[1] = 4; exige n >= 2
```

Se a entrada for `3 8 4 9`, o vetor fica `[8, 4, 9]` e o valor da posição 2 do enunciado é `4`.

Para ler uma matriz com `linhas` e `colunas` já conhecidas:

```java
int linhas = 2, colunas = 3;
int[][] grade = new int[linhas][colunas];
for (int i = 0; i < linhas; i++) {
    for (int j = 0; j < colunas; j++) {
        grade[i][j] = entrada.nextInt();
    }
}
// Com a entrada 1 2 3 4 5 6: grade = {{1, 2, 3}, {4, 5, 6}} e grade[1][0] = 4
```

```java
String texto = "banana";
int tamanho = texto.length();     // 6
char letra = texto.charAt(0);      // 'b'
String pedaco = texto.substring(1, 4); // "ana", fim excluído
boolean igual = texto.equals("banana");  // true
boolean temAna = texto.contains("ana");  // true
int posicao = texto.indexOf('n');  // 2; devolve -1 se não encontrar
```

Use `v.length` para vetor, `texto.length()` para string e `lista.size()` para coleção. Compare o conteúdo de strings com `.equals()`. Para letras de `a` a `z`, `letra - 'a'` produz um índice de 0 a 25.

6. Métodos

Declare métodos dentro da classe, fora de `main`. Use `static` para chamar diretamente a partir do `main`.

```java
static int contarPares(int[] valores) {
    int quantidade = 0;
    for (int valor : valores) {
        if (valor % 2 == 0) {
            quantidade++;
        }
    }
    return quantidade;
}
```

No `main`, chame com `int resposta = contarPares(new int[]{2, 7, 4});`. Dentro do método, `quantidade` passa por 1, 1 e 2, então `resposta` vale `2`. O tipo antes do nome é o tipo retornado. Um método `void` executa ações sem retornar um valor. Se um método alterar uma posição do vetor recebido, o chamador verá essa alteração.

7. Biblioteca que vale decorar

Adicione os imports correspondentes, como `import java.util.Arrays;`. Nas coleções genéricas, escreva `Integer` ou `Long` para números inteiros.

```java
int[] v = {7, 2, 4};
Arrays.sort(v);                          // {2, 4, 7}
int indice = Arrays.binarySearch(v, 4);  // 1; negativo se ausente
Arrays.fill(v, -1);                      // {-1, -1, -1}

ArrayList<Integer> lista = new ArrayList<>();
lista.add(10);                           // [10]
lista.add(20);                           // [10, 20]
int item = lista.get(0);                 // 10
lista.set(0, 30);                        // [30, 20]
lista.sort(null);                        // [20, 30], ordem crescente

HashSet<Integer> vistos = new HashSet<>();
vistos.add(8);                           // {8}
boolean repetido = vistos.contains(8);   // true

HashMap<Integer, Integer> frequencia = new HashMap<>();
int valor = 8;
frequencia.put(valor, frequencia.getOrDefault(valor, 0) + 1); // {8=1}

ArrayDeque<Integer> fila = new ArrayDeque<>();
fila.addLast(10);                        // [10]
fila.addLast(20);                        // [10, 20]
int primeiro = fila.removeFirst();       // 10; fila = [20]

ArrayDeque<Integer> pilha = new ArrayDeque<>();
pilha.push(10);                          // [10]
pilha.push(20);                          // [20, 10], topo à esquerda
int topo = pilha.pop();                  // 20; pilha = [10]

PriorityQueue<Integer> menores = new PriorityQueue<>();
menores.add(20);
menores.add(10);
int menor = menores.remove();            // 10; resta 20
```

Use `isEmpty()` antes de remover se a coleção puder estar vazia. Para desempatar, faça a comparação explicitamente. `HashMap` e `HashSet` não garantem a ordem de iteração.

8. Padrões de prova

Quando a questão pede...	Experimente...	Custo
Maior valor ou contagem	Percorrer uma vez, atualizando a resposta	O(N)
Frequência com valores de 0 a 100	`int[] cont = new int[101];` e `cont[x]++;`	O(N + 101)
Mediana	Ordenar e acessar `v[v.length / 2]` se N for ímpar	O(N log N)
Muitas somas em intervalos fixos	Pré-calcular prefixos	O(N + Q)
Descobrir se um valor já apareceu	Guardar valores em `HashSet`	O(N) em média

Com N = Q = 100.000, somar todos os valores de cada consulta pode chegar a 10 bilhões de operações. Para uma consulta com posições de 1 a N, use:

```java
long[] prefixo = new long[n + 1];  // n = 3: [0, 0, 0, 0]
for (int i = 1; i <= n; i++) {
    prefixo[i] = prefixo[i - 1] + entrada.nextInt(); // lê 2, -1, 4
}
// prefixo = [0, 2, 1, 5]
long soma = prefixo[r] - prefixo[l - 1]; // l = 2, r = 3: 5 - 2 = 3
```

`prefixo[0]` vale zero. Em `[2, -1, 4]`, os prefixos são `[0, 2, 1, 5]`. A soma de 2 até 3 é `5 - 2 = 3`. O programa completo, com leitura rápida, está no gabarito de Somas de intervalos (Parte 2).

9. Antes de submeter

1. Rode o exemplo da questão.
2. Crie um caso mínimo, um caso de empate e um caso no limite dos valores, se forem possíveis.
3. Confira se o enunciado numera as posições a partir de 1 e se o seu vetor começa em 0.
4. Estime o maior valor de uma soma ou produto antes de escolher entre `int` e `long`.
5. Confira nome da classe e do arquivo, formato da saída e ausência de mensagens de depuração.

Sintoma	O que verificar
`cannot find symbol`	Nome digitado, escopo e imports
`ArrayIndexOutOfBoundsException`	Se o índice está entre 0 e `length - 1`
`InputMismatchException`	Se o leitor espera o tipo correto e se um inteiro cabe no tipo
Linha vazia após `nextInt()`	Se `nextLine()` está lendo o fim da linha anterior
Resposta errada	Empate, negativos, limites inclusivos e overflow
Tempo excedido	Complexidade, volume de entrada e quantidade de chamadas de impressão

Volte aos exercícios (Parte 2) e tente reescrever cada solução de memória depois de entender o gabarito.

Parte 2 — Exercícios

Resolva no papel primeiro. Depois digite, rode o exemplo e só então confira o gabarito.

Em todos os exercícios, leia da entrada padrão e imprima apenas a resposta.

Dia 1 — Recordes

Leia N (1 a 10.000) e N temperaturas inteiras entre -1.000 e 1.000. Conte quantas são estritamente maiores que todas as anteriores. A primeira sempre conta.

- Entrada: N = 6, temperaturas = -3 -3 -1 -2 0 0. Saída: 3.
- Teste também uma temperatura só e uma lista com todos os valores iguais.
- Dica: guarde o maior valor visto até agora.

Sua solução / rascunho

Dia 1 — Maior sequência

Leia N (1 a 10.000) e N inteiros. Imprima o comprimento do maior trecho consecutivo em que todos os valores são iguais.

- Entrada: N = 7, valores = 4 4 2 4 4 4 2. Saída: 3.
- Teste também uma lista em que o maior trecho termina no último elemento.
- Dica: acompanhe o tamanho do trecho atual e o maior tamanho encontrado.

Sua solução / rascunho

Dia 2 — Mediana

Leia N (ímpar, de 1 a 10.000) e N inteiros. Imprima o valor que fica no meio depois de ordenar os números em ordem crescente.

- Entrada: N = 5, valores = 9 -2 9 1 3. Saída: 3.
- Dica: depois de ordenar, qual é o índice do meio se o primeiro é zero?

Sua solução / rascunho

Dia 2 — Frequências

Leia N (1 a 10.000) e N inteiros entre 0 e 100. Imprima o valor que mais aparece. Em caso de empate, imprima o menor.

- Entrada: N = 6, valores = 3 1 3 1 5 0. Saída: 1.
- Dica: use o próprio valor como índice de um vetor de contadores.

Sua solução / rascunho

Dia 3 — Somas de intervalos

Leia N e Q (cada um de 1 a 100.000), depois N inteiros entre -1.000.000.000 e 1.000.000.000. Em seguida, leia Q pares L e R (1 ≤ L ≤ R ≤ N). Para cada par, imprima a soma dos valores entre as posições L e R, inclusive, em uma linha.

```
Entrada
5 3
2 -1 4 3 2
1 3
2 5
4 4

Saída
5
8
3
```

- Dica: calcule uma vez a soma dos primeiros i elementos para cada i. Use long para as somas.
- Teste um intervalo que começa em 1 e outro que contém dois valores de 1.000.000.000 seguidos de mais um positivo.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema I

Leia 8 números, cada um 0, 1 ou 9. Imprima `S` se nenhum for 9, ou `F` se algum for 9.

- Entrada: 0 0 1 9 0 1 0 1. Saída: F.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema A

Leia N e uma palavra só com `a` e `b`. Some o tamanho dos blocos de `a` seguidos que têm 2 letras ou mais.

- Entrada: N = 10, palavra = aababaaabb. Saída: 5 (blocos `aa` e `aaa`).
- Dica: é parecido com Maior sequência.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema J

Jogo de 23: cada jogador soma as próprias 2 cartas e as cartas da mesa. Cartas 11, 12 e 13 valem 10, e cada carta tem 4 cópias. Imprima a menor carta que faz Maria vencer na próxima rodada, ou -1. Maria vence se chegar a 23, ou se João passar de 23 e ela ficar abaixo.

- Entrada: N = 2, João = 8 4, Maria = 4 1, mesa = 4 4. Saída: 5.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema E

Balões em fila com alturas H. Uma flecha estoura um balão da sua altura e desce 1. Imprima o menor número de flechas.

- Entrada: N = 5, alturas = 3 2 1 5 4. Saída: 2.
- Dica: guarde quantas flechas estão voando em cada altura.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema D

Leia N, x e y. Imprima quantas ativações levam a partícula até (x, y).

- Entrada: 4 12 4. Saída: 1.
- Dica: olhe quantos zeros x tem no fim, em binário.

Sua solução / rascunho

Dia 4 — Maratona 2022, problema F

N palavras de tamanho C, cada uma com um `*`. Ache a palavra sem `*` que combina com mais palavras. Em caso de empate, imprima a menor em ordem alfabética.

- Entrada: rat* ru*d rot* r*ta r*ta. Saída: rata 3.
- Dica: troque o `*` por cada letra de `a` a `z` e conte com um `HashMap`.

Sua solução / rascunho

Gabaritos

Os comentários seguem a entrada de exemplo de cada exercício.

Gabarito — Recordes `Recordes.java`

```java
import java.util.Scanner;

// Valores comentados para a entrada do exercício: 6 / -3 -3 -1 -2 0 0
public class Recordes {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();        // n = 6
        int maior = entrada.nextInt();    // maior = -3
        int recordes = 1;                 // recordes = 1, a primeira sempre conta

        for (int i = 1; i < n; i++) {
            int temperatura = entrada.nextInt(); // -3, -1, -2, 0, 0
            if (temperatura > maior) {
                maior = temperatura;      // maior: -1 (i = 2), 0 (i = 4)
                recordes++;               // recordes: 2 (i = 2), 3 (i = 4)
            }
        }

        System.out.println(recordes);     // imprime 3
    }
}
```

Gabarito — Maior sequência `MaiorSequencia.java`

```java
import java.util.Scanner;

// Valores comentados para a entrada do exercício: 7 / 4 4 2 4 4 4 2
public class MaiorSequencia {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();         // n = 7
        int anterior = entrada.nextInt();  // anterior = 4
        int atual = 1;                     // tamanho do trecho atual
        int maior = 1;                     // maior trecho visto

        for (int i = 1; i < n; i++) {
            int valor = entrada.nextInt(); // 4, 2, 4, 4, 4, 2
            if (valor == anterior) {
                atual++;
            } else {
                atual = 1;
            }
            // atual: 2, 1, 1, 2, 3, 1
            maior = Math.max(maior, atual);
            // maior: 2, 2, 2, 2, 3, 3
            anterior = valor;
        }

        System.out.println(maior);         // imprime 3
    }
}
```

Gabarito — Mediana `Mediana.java`

```java
import java.util.Arrays;
import java.util.Scanner;

// Valores comentados para a entrada do exercício: 5 / 9 -2 9 1 3
public class Mediana {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();          // n = 5
        int[] valores = new int[n];

        for (int i = 0; i < n; i++) {
            valores[i] = entrada.nextInt();
        }
        // valores = [9, -2, 9, 1, 3]

        Arrays.sort(valores);               // valores = [-2, 1, 3, 9, 9]
        System.out.println(valores[n / 2]); // n / 2 = 2, imprime 3
    }
}
```

Gabarito — Frequências `Frequencias.java`

```java
import java.util.Scanner;

// Valores comentados para a entrada do exercício: 6 / 3 1 3 1 5 0
public class Frequencias {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();          // n = 6
        int[] contagem = new int[101];      // índices de 0 a 100, todos 0

        for (int i = 0; i < n; i++) {
            int valor = entrada.nextInt(); // 3, 1, 3, 1, 5, 0
            contagem[valor]++;
        }
        // contagem[0] = 1, contagem[1] = 2, contagem[3] = 2, contagem[5] = 1

        int resposta = 0;                   // começa no valor 0, que aparece 1 vez
        for (int valor = 1; valor < contagem.length; valor++) {
            // Se houver empate, mantemos o menor valor.
            if (contagem[valor] > contagem[resposta]) {
                resposta = valor;           // vira 1 (2 > 1); em 3, 2 > 2 é falso
            }
        }

        System.out.println(resposta);       // imprime 1
    }
}
```

Gabarito — Somas de intervalos `SomasIntervalos.java`

```java
import java.io.BufferedInputStream;
import java.io.EOFException;
import java.io.IOException;

// Valores comentados para a entrada do exercício:
// 5 3 / 2 -1 4 3 2 / 1 3 / 2 5 / 4 4
public class SomasIntervalos {
    public static void main(String[] args) throws IOException {
        BufferedInputStream entrada = new BufferedInputStream(System.in);
        int n = lerInt(entrada);          // n = 5
        int consultas = lerInt(entrada);  // consultas = 3

        long[] prefixo = new long[n + 1]; // [0, 0, 0, 0, 0, 0]
        for (int i = 1; i <= n; i++) {
            prefixo[i] = prefixo[i - 1] + lerInt(entrada); // lê 2, -1, 4, 3, 2
        }
        // prefixo = [0, 2, 1, 5, 8, 10]

        StringBuilder saida = new StringBuilder();
        for (int i = 0; i < consultas; i++) {
            int esquerda = lerInt(entrada); // 1, 2, 4
            int direita = lerInt(entrada);  // 3, 5, 4
            saida.append(prefixo[direita] - prefixo[esquerda - 1]).append('\n');
            // 5 - 0 = 5, 10 - 2 = 8, 8 - 5 = 3
        }
        System.out.print(saida);          // imprime 5, 8 e 3, um por linha
    }

    // Lê um inteiro com sinal opcional em uma entrada válida de competição.
    static int lerInt(BufferedInputStream entrada) throws IOException {
        int caractere = entrada.read();
        while (caractere != -1 && caractere <= ' ') {
            caractere = entrada.read();
        }
        if (caractere == -1) {
            throw new EOFException("Fim da entrada antes do próximo inteiro");
        }

        boolean negativo = caractere == '-';
        if (negativo || caractere == '+') {
            caractere = entrada.read();
        }

        // Acumula em long para comportar o módulo de Integer.MIN_VALUE.
        long valor = 0;
        // Para o texto "-12": negativo = true; valor passa por 1 e 12; retorna -12.
        while (caractere >= '0' && caractere <= '9') {
            valor = valor * 10 + caractere - '0';
            caractere = entrada.read();
        }
        return (int) (negativo ? -valor : valor);
    }
}
```

Gabarito — Maratona 2022, problema I `I.java`

```java
import java.util.Scanner;

// Problema I — Interceptando Informações
// Valores comentados para a entrada: 0 0 1 9 0 1 0 1
public class I {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        boolean falhou = false;
        for (int i = 0; i < 8; i++) {
            int bit = entrada.nextInt(); // 0, 0, 1, 9, 0, 1, 0, 1
            if (bit == 9) {
                falhou = true;           // vira true no 4º valor
            }
        }
        System.out.println(falhou ? "F" : "S"); // imprime F
    }
}
```

Gabarito — Maratona 2022, problema A `A.java`

```java
import java.util.Scanner;

// Problema A — Achando os Monótonos Não-Triviais Maximais
// Valores comentados para a entrada: 10 / aababaaabb
public class A {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();      // n = 10
        String s = entrada.next();      // s = "aababaaabb"

        long resposta = 0;
        int inicio = 0;
        while (inicio < n) {
            int fim = inicio;
            while (fim < n && s.charAt(fim) == s.charAt(inicio)) {
                fim++;
            }
            int tamanho = fim - inicio; // blocos: aa(2) b(1) a(1) b(1) aaa(3) bb(2)
            if (s.charAt(inicio) == 'a' && tamanho >= 2) {
                resposta += tamanho;    // resposta: 2, depois 5
            }
            inicio = fim;               // pula para o próximo bloco
        }

        System.out.println(resposta);   // imprime 5
    }
}
```

Gabarito — Maratona 2022, problema J `J.java`

```java
import java.util.Scanner;

// Problema J — Jogando 23
// Cada carta (1 a 13) tem 4 cópias. Cartas 11, 12 e 13 valem 10.
// Maria vence se chegar a 23, ou se João passar de 23 e ela ficar abaixo.
// Entrada de exemplo: 2 / 8 4 / 4 1 / 4 4
public class J {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();
        int[] usadas = new int[14];
        int joao = 0, maria = 0;

        for (int i = 0; i < 2; i++) {          // cartas de João: 8, 4
            int c = entrada.nextInt();
            usadas[c]++;
            joao += Math.min(c, 10);
        }
        for (int i = 0; i < 2; i++) {          // cartas de Maria: 4, 1
            int c = entrada.nextInt();
            usadas[c]++;
            maria += Math.min(c, 10);
        }
        for (int i = 0; i < n; i++) {          // cartas da mesa: 4, 4 (contam para os dois)
            int c = entrada.nextInt();
            usadas[c]++;
            joao += Math.min(c, 10);
            maria += Math.min(c, 10);
        }
        // joao = 20, maria = 13

        for (int c = 1; c <= 13; c++) {
            if (usadas[c] == 4) continue;      // o 4 acabou
            int j = joao + Math.min(c, 10);
            int m = maria + Math.min(c, 10);
            if (m == 23 || (j > 23 && m < 23)) {
                System.out.println(c);         // imprime 5
                return;
            }
        }
        System.out.println(-1);
    }
}
```

Gabarito — Maratona 2022, problema E `E.java`

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

// Problema E — Estourando Balões
// Entrada de exemplo: 5 / 3 2 1 5 4
public class E {
    public static void main(String[] args) throws Exception {
        // BufferedReader é mais rápido que Scanner para muitos números
        BufferedReader leitor = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(leitor.readLine().trim());     // n = 5
        StringTokenizer partes = new StringTokenizer(leitor.readLine());

        int[] flechas = new int[1_000_002]; // flechas[h] = flechas voando na altura h
        int resposta = 0;

        for (int i = 0; i < n; i++) {
            int h = Integer.parseInt(partes.nextToken());       // 3, 2, 1, 5, 4
            if (flechas[h] > 0) {
                flechas[h]--;      // já tem flecha nessa altura
            } else {
                resposta++;        // precisa de flecha nova (balões 3 e 5)
            }
            flechas[h - 1]++;      // a flecha continua uma altura abaixo
        }

        System.out.println(resposta);                           // imprime 2
    }
}
```

Gabarito — Maratona 2022, problema D `D.java`

```java
import java.util.Scanner;

// Problema D — Deslocando Partículas
// Cada ativação "corta pela metade". Depois de k ativações, x termina com N-1-k zeros em binário.
// Então k = N - 1 - (zeros no fim de x).
// Entrada de exemplo: 4 12 4
public class D {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();                                  // n = 4
        int x = entrada.nextInt();                                  // x = 12 (1100 em binário)
        int zerosNoFim = Integer.numberOfTrailingZeros(x);          // 2
        System.out.println(n - 1 - zerosNoFim);                     // 4 - 1 - 2 = 1
    }
}
```

Gabarito — Maratona 2022, problema F `F.java`

```java
import java.util.HashMap;
import java.util.Scanner;

// Problema F — Forca Multidimensional
// Ideia: troque o '*' de cada palavra por cada letra de 'a' a 'z' e conte
// quantas palavras do tabuleiro combinam com o resultado.
// Entrada de exemplo: 5 4 / rat* ru*d rot* r*ta r*ta
public class F {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        int n = entrada.nextInt();
        int c = entrada.nextInt();
        String[] palavras = new String[n];
        HashMap<String, Integer> quantas = new HashMap<>();
        for (int i = 0; i < n; i++) {
            palavras[i] = entrada.next();
            quantas.put(palavras[i], quantas.getOrDefault(palavras[i], 0) + 1);
        }
        // quantas = {rat*=1, ru*d=1, rot*=1, r*ta=2}

        String melhor = "";
        int melhorConta = 0;

        for (String palavra : palavras) {
            for (char letra = 'a'; letra <= 'z'; letra++) {
                String p = palavra.replace('*', letra);          // ex.: "rata"

                // p combina com cada palavra igual a p com uma letra virada '*'
                int conta = 0;
                for (int j = 0; j < c; j++) {
                    String comEstrela = p.substring(0, j) + "*" + p.substring(j + 1);
                    conta += quantas.getOrDefault(comEstrela, 0);
                }
                // "rata": "rat*" (1) + "r*ta" (2) = 3

                if (conta > melhorConta || (conta == melhorConta && p.compareTo(melhor) < 0)) {
                    melhor = p;
                    melhorConta = conta;
                }
            }
        }

        System.out.println(melhor + " " + melhorConta);        // imprime "rata 3"
    }
}
```

Folha de consulta rápida

Esqueleto

```
import java.util.*;

public class Main {
  public static void main(String[] a) {
    Scanner in = new Scanner(System.in);
    int n = in.nextInt();
    // ...
    System.out.println(resp);
  }
}
```

Leitura

`in.nextInt()`	int
`in.nextLong()`	long
`in.nextDouble()`	decimal (use Locale.US)
`in.next()`	uma palavra
`in.nextLine()`	resto da linha
`in.hasNextInt()`	ainda há inteiro?

Saída

```
System.out.println(x);
System.out.print(x + " ");
System.out.printf(Locale.US, "%.2f%n", d);
StringBuilder sb = new StringBuilder();
sb.append(x).append('\n');
System.out.print(sb);
```

Limites

`int`	até ~2,1 × 10^9
`long`	até ~9,2 × 10^{18}
Produto	`1L * a * b`

Vetores e strings

```
int[] v = new int[n];      // zeros
v.length    s.length()    lista.size()
Arrays.sort(v);
Arrays.fill(v, -1);
s.charAt(i)   s.substring(i, j)
s.equals(t)   s.indexOf('x')
c - 'a'       // 0..25
```

Coleções

```
ArrayList<Integer> l = new ArrayList<>();
l.add(x); l.get(i); l.set(i, x);
HashSet<Integer> s = new HashSet<>();
s.add(x); s.contains(x);
HashMap<Integer,Integer> f = new HashMap<>();
f.put(x, f.getOrDefault(x, 0) + 1);
ArrayDeque<Integer> q = new ArrayDeque<>();
q.addLast(x); q.removeFirst(); // fila
q.push(x);    q.pop();         // pilha
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(x); pq.remove();        // menor
```

Quanto cabe no tempo (~10^8 op/s)

N até	Complexidade
10–20	O(2^N), testar tudo
~500	O(N^3)
~5.000	O(N^2)
~10^6	O(N log N), O(N)

Prefixos

```
p[i] = p[i-1] + v[i];      // p[0] = 0
soma(l..r) = p[r] - p[l-1];
```

Registro de tentativas

Depois de cada tentativa, anote a ideia, a complexidade e um caso que você esqueceu de testar.

Questão	Data	Ideia	Complexidade	Caso esquecido

Checklist antes de submeter

- ☐ Rode o exemplo da questão.
- ☐ Crie um caso mínimo, um caso de empate e um caso no limite dos valores, se forem possíveis.
- ☐ Confira se o enunciado numera as posições a partir de 1 e se o seu vetor começa em 0.
- ☐ Estime o maior valor de uma soma ou produto antes de escolher entre `int` e `long`.
- ☐ Confira nome da classe e do arquivo, formato da saída e ausência de mensagens de depuração.